

Web Systems Application Development

Jason Kuruzovich

kuruzj@rpi.edu



Agenda

1. Introductions
2. Course
3. History of the Web
4. Next Steps and Technical Setup

Introductions



Introductions



Your name—and what you prefer to be called



When you're not in class, what do you like to do?



After graduation: dream job or vague direction

The Courses

**Goal: Create a Technical
Course Sequence that
Prepares you to Create and
Manage the Full Landscape
of Web Applications**

THE WEB SYSTEMS PATHWAY

INTRO

LAMP FOUNDATIONS



Linux • Apache • MySQL • PHP

WEB SYSTEMS

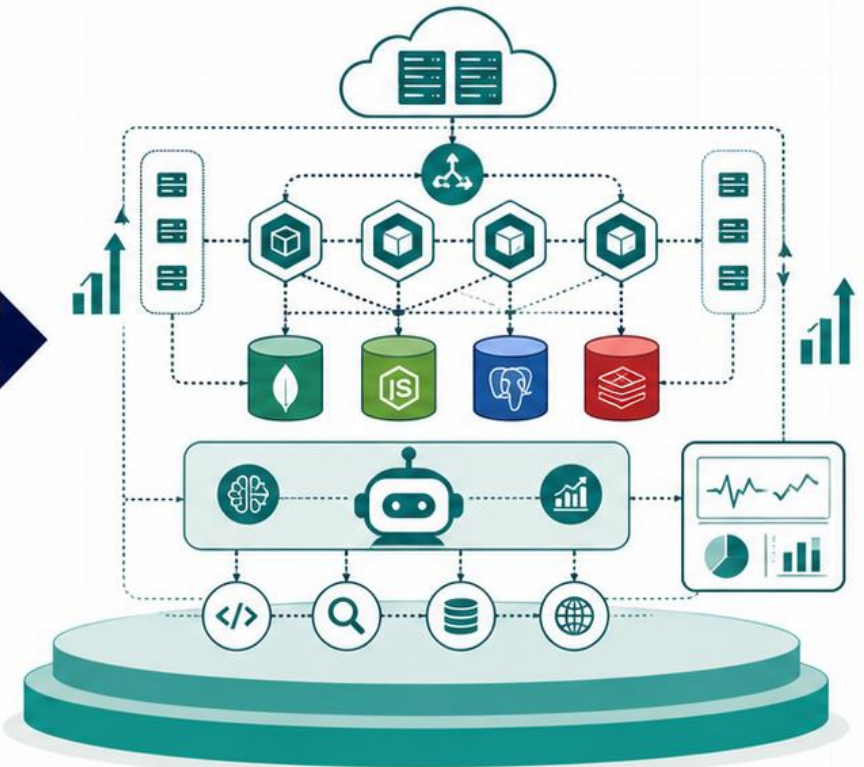
MODERN FULL STACK



PHP • React • Tailwind / Bootstrap

ADVANCED WEB SYSTEMS

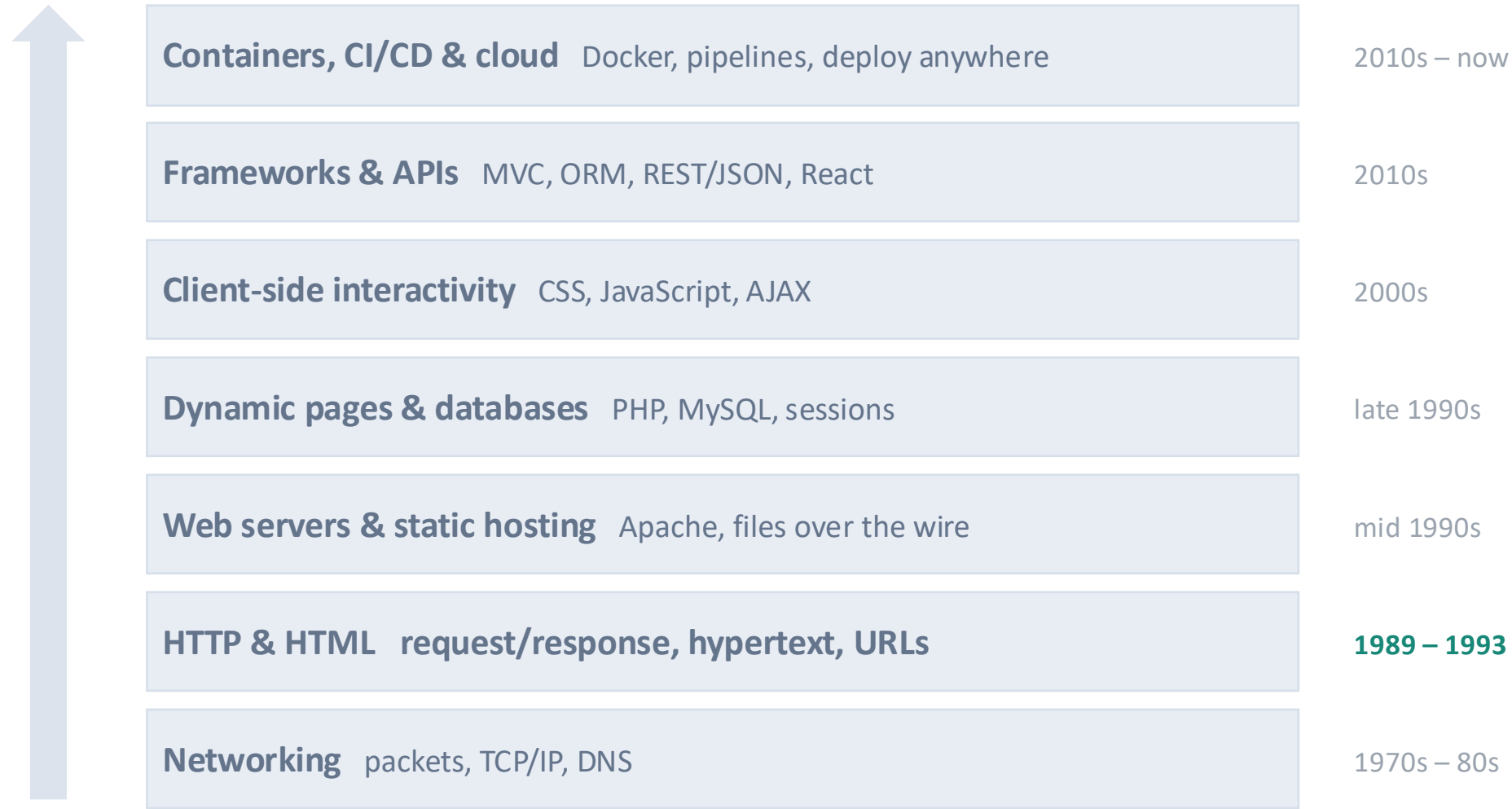
ARCHITECTURE AT SCALE



Advanced Frameworks • MERN • Agentic AI • Scale

Web Systems: The web systems stack, built layer by layer

Each layer assumes the one below it — and each was invented to solve a problem the layer below created



Where are you with Web Systems Development?

I could do this today

I want to know more

Learning Goals

By the end of the course, students will be able to:

1. Explain how a full-stack web application works—from the browser to the server, database, and network.
2. Select and justify appropriate technologies, architectures, and design patterns.
3. Build effective front-end experiences using HTML, CSS, JavaScript, and progressive enhancement.
4. Develop, integrate, and troubleshoot application back ends and web services.
5. Identify common web security risks and apply protections such as input validation, secure authentication, authorization, and safe data handling.
6. Explain how AI is used in web development and applications, critically evaluate its outputs, and recognize its capabilities, limitations, and security implications.
7. Plan, build, test, deploy, and present a complete web application using modern development practices.

Schedule

Tuesday

- Lecture
- Discussion
- Live demonstration
- Guest Speakers

Friday

- Lab/Homework
- Group project collaboration

Late Policy

- Late submissions are accepted for **one week** following the due date with a **10% grade reduction**. Submissions more than one week late receive a **30% reduction**.
- No submissions are accepted after **Friday, December 11** — the last day of classes. The final project report is due during the exam period and is not subject to this cutoff.
- **In-class assessments cannot be submitted late.** A in class quiz or presentation missed without a prior approved excuse is graded zero. Excused absences are arranged in advance; see the attendance policy.

Participation

- This course works best when we build it together: the value of the class is something we co-create
- Many of you are stronger web developers than I am; share what you know and help your peers
- See the attendance sheet and participation guidelines (format adapted from Professor Langer)

Course Assessment Measures

Assessment	Weight
Lab/Homework Assignments	30%
Semester Group Project	30%
– Overall System Architecture & Performance	15%
– Individual Contribution & Professional Practice*	15%
Quizzes	30%
Participation	10%
Total	100%

Tentative Course Calendar (1 of 3)

Fall 2026 · ITWS 2110 · Aug 27 – Dec 11, 2026 · Tuesday/Friday · 28 sessions

#	Date	Session	Due 11:59 PM
1	Fri 8/28	Course intro; the modern web application. How grading works: clone, run the suite, red to green	
2	Tue 9/1	Docker and infrastructure as code	
3	Fri 9/4	LAMP stack on Docker; introduction to GitHub Classroom	Lab 0: Environment + repo
4	Tue 9/8	Test-driven development — tests as specification; Pest/PHPUnit anatomy	
5	Fri 9/11	Writing tests; CI runs your suite on every push · teams form · last day to add/drop	Lab 1: Dockerized LAMP stack
6	Tue 9/15	CSS frameworks I — Bootstrap / Tailwind; utility vs. component approaches	
7	Fri 9/18	CSS frameworks II — responsive layout, design systems; when not to use one	Lab 2: Testing infrastructure
8	Tue 9/22	React I — components, props, the render model	
9	Fri 9/25	React II — state, events, forms · project proposal	Lab 3: CSS framework build
10	Tue 9/29	React III — lists, lifting state, composition	

Tentative Course Calendar (2 of 3)

Sessions 11–18 · October

#	Date	Session	Due 11:59 PM
11	Fri 10/2	React IV — component testing with Vitest and Testing Library	Lab 4: React components
12	Tue 10/6	Application frameworks: Laravel I — what a framework buys you; MVC, routing, Blade	
13	Fri 10/9	Laravel II — controllers, requests, HTTP tests · Quiz 1 review	Lab 5: React with component tests
14	Tue 10/13	QUIZ 1 — handwritten, closed-device (covers sessions 1–11)	
—	<i>Fri 10/16</i>	<i>No class — follows a Monday schedule</i>	
15	Tue 10/20	ORM I — Eloquent, migrations, seeders; schema as code	
16	Fri 10/23	ORM II — relationships, eager loading, the N+1 problem; database testing	Lab 6: Laravel app (2-wk)
17	Tue 10/27	Forms, validation, sessions, flash messages, CSRF tokens	
18	Fri 10/30	Authentication — authn vs. authz, password storage, Breeze, middleware, policies	Lab 7: Data model + ORM

Tentative Course Calendar (3 of 3)

Sessions 19–25 · November – December

#	Date	Session	Due 11:59 PM
19	Tue 11/3	SECURITY — OWASP Top 10:2025; injection, XSS, CSRF; headers, TLS, secrets, supply-chain risk · project midpoint · last day to drop with W	
20	Fri 11/6	Security Lab	Lab 8: Forms + authentication
21	Tue 11/10	APIs — JSON, REST, resource design, versioning, documentation, contract testing; Sanctum vs. session auth	
22	Fri 11/13	API Lab — wire your React front end to your Laravel API	Lab 9: Security hardening + findings
23	Tue 11/17	Cloud deployment — build, ship, run; environments, secrets, migrations in production	
24	Fri 11/20	Deployment Lab	Lab 10: JSON API + React consuming it
—	11/23–27	<i>Thanksgiving Break — no classes</i>	
25	Tue 12/1	Performance — caching layers, revisiting N+1, load testing, measuring before/after	

AI Policy

Every lab states its own AI rules on the assignment page — read them before you start.

Some exercises are AI-assisted by design; others you do unaided.

Where AI helps

- Explaining errors and unfamiliar code
- Debugging environment and config problems
- Boilerplate, scaffolding, config files
- Drafting docs, comments, commit messages
- Generating sample or test data

Where you work unaided

- Quizzes — handwritten, closed-device
- Exercises marked “no AI” on the lab page
- Where you are asked to “write in your own words”

Discussion

What are the characteristics of good assignments in an age of AI?

 The 7 • LIVE

AI can now credibly complete most undergraduate assignments, MIT warns

A new report urges rapid changes to college teaching, learning and assessment as the technology advances.

Yesterday at 5:39 p.m. EDT

History of Web Systems

Web systems have profoundly reshaped society



Commerce



Health



Education



News & media

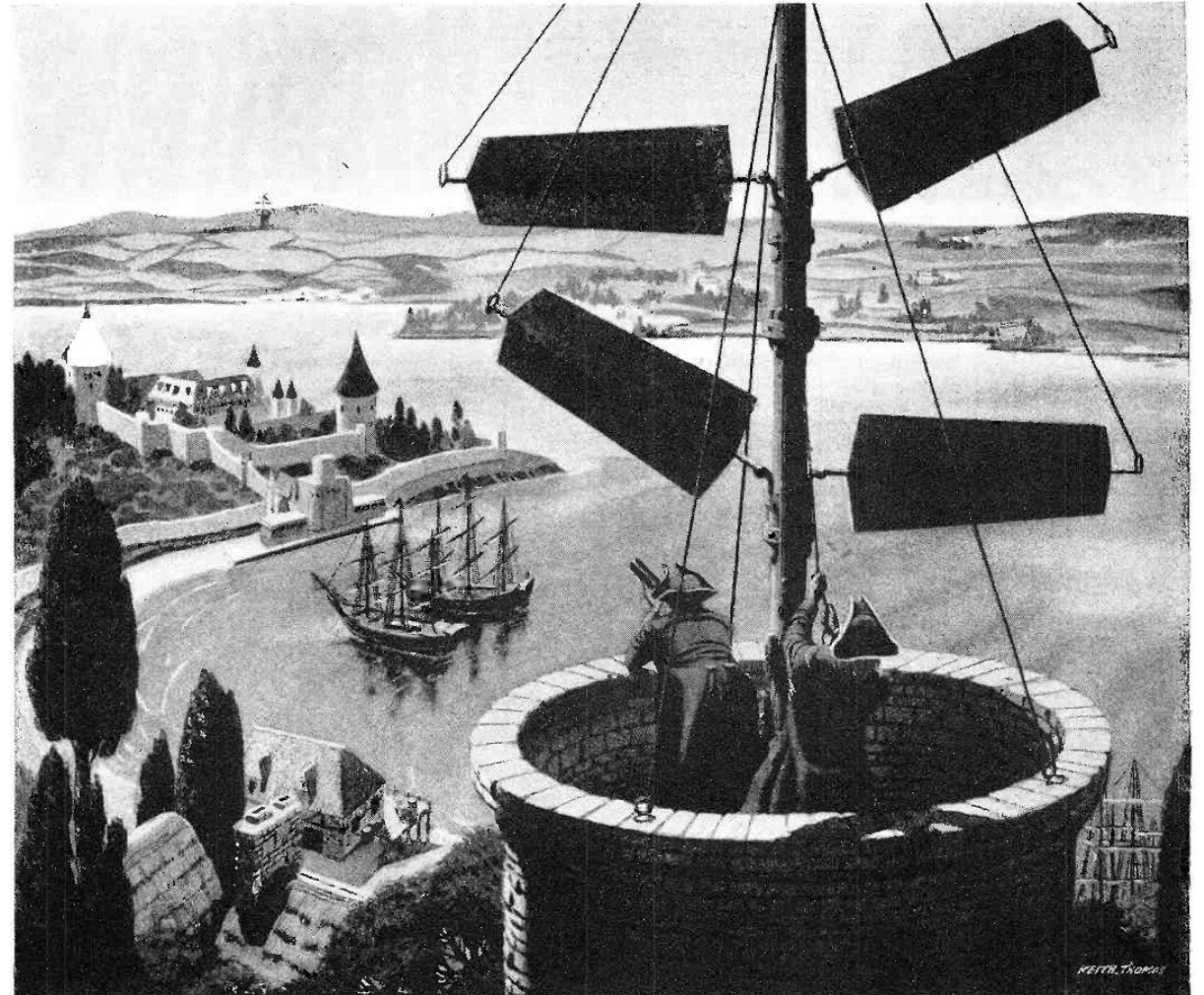


Community

Internet 0.1 Beta (18th Century)

Semaphore Telegraph

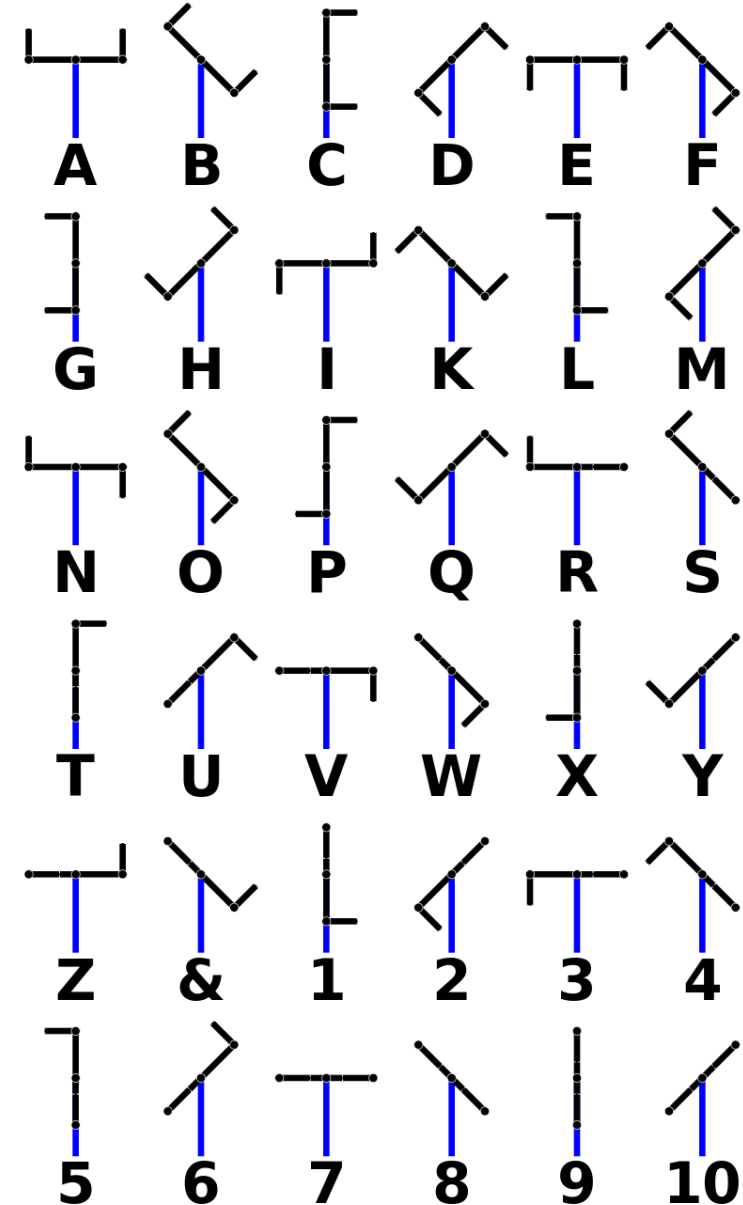
- Visual texting by position of the mechanical elements;



By The drawing is signed "Keith Thomas" in lower right corner [Public domain], via Wikimedia Commons

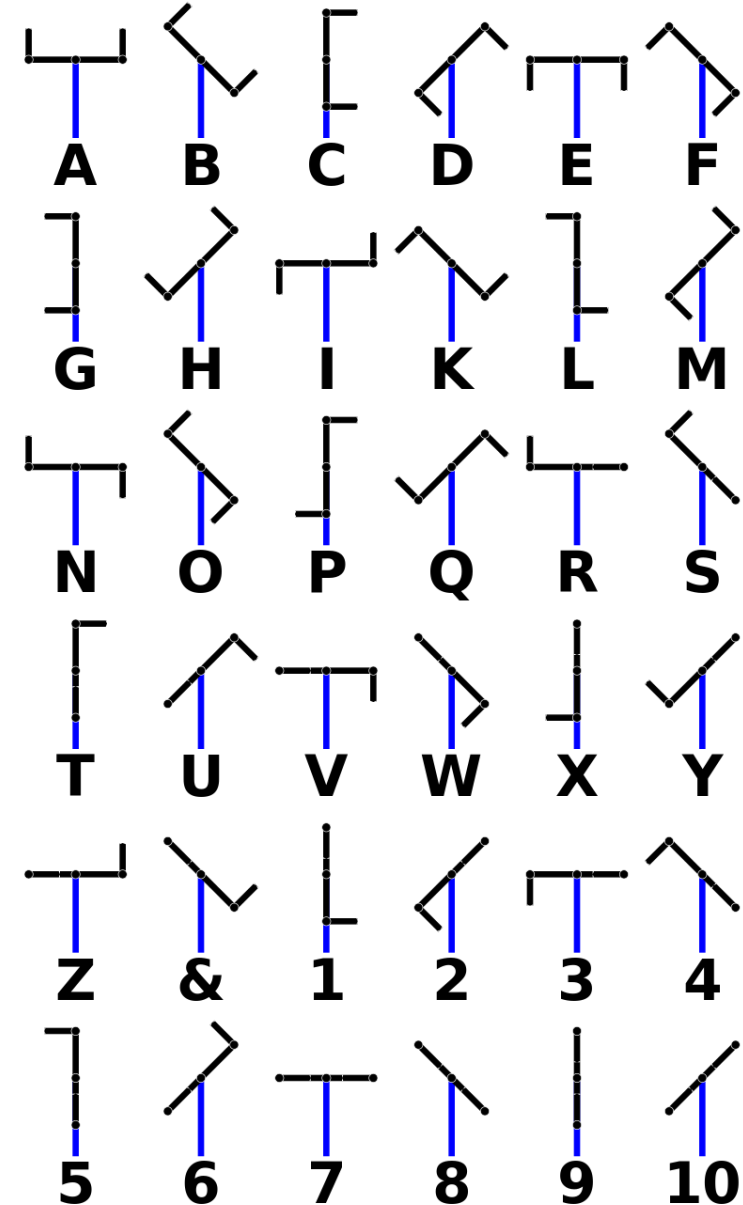
Internet 0.1 Beta (18th Century)

- Chappe system, as used simply for signaling letters and numbers.



Internet 0.1 Beta (18th Century)

- Over 50 stations connected France.
- A shared visual code moved messages across the network.

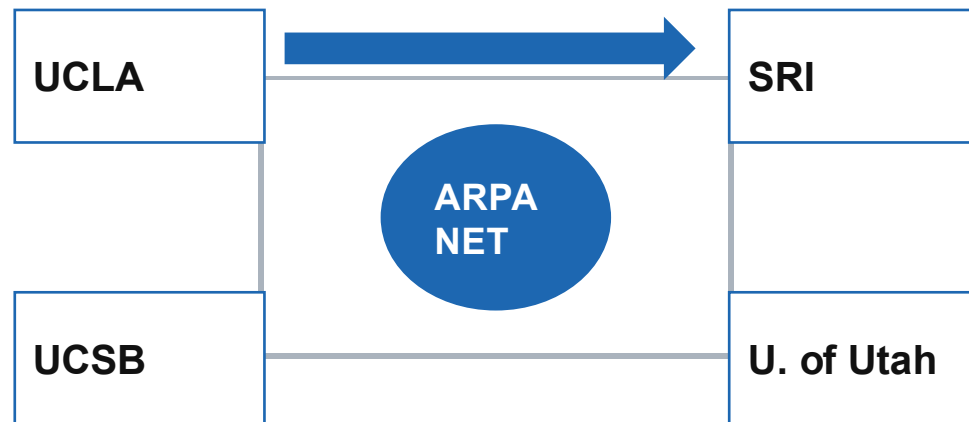


ARPANET: the network that became the Internet's foundation

Funded by the U.S. Advanced Research Projects Agency, ARPANET proved that packet-switched computers could communicate across long distances.

THE FIRST NETWORK

Four initial nodes by December 1969



“LO” — first host-to-host message, UCLA → SRI
October 29, 1969

FROM EXPERIMENT TO INTERNET

- 1969** ARPANET begins operation; the first message travels between UCLA and SRI.
- 1983** The network switches from NCP to TCP/IP—the protocol suite of the modern Internet.
- 1990** ARPANET is formally decommissioned after other networks carry its legacy forward.

Key idea ARPANET did not equal today’s Internet—but its packet switching, protocols, and network-of-networks concept supplied the technical foundation.

How the network evolved: packet switching to IPv6

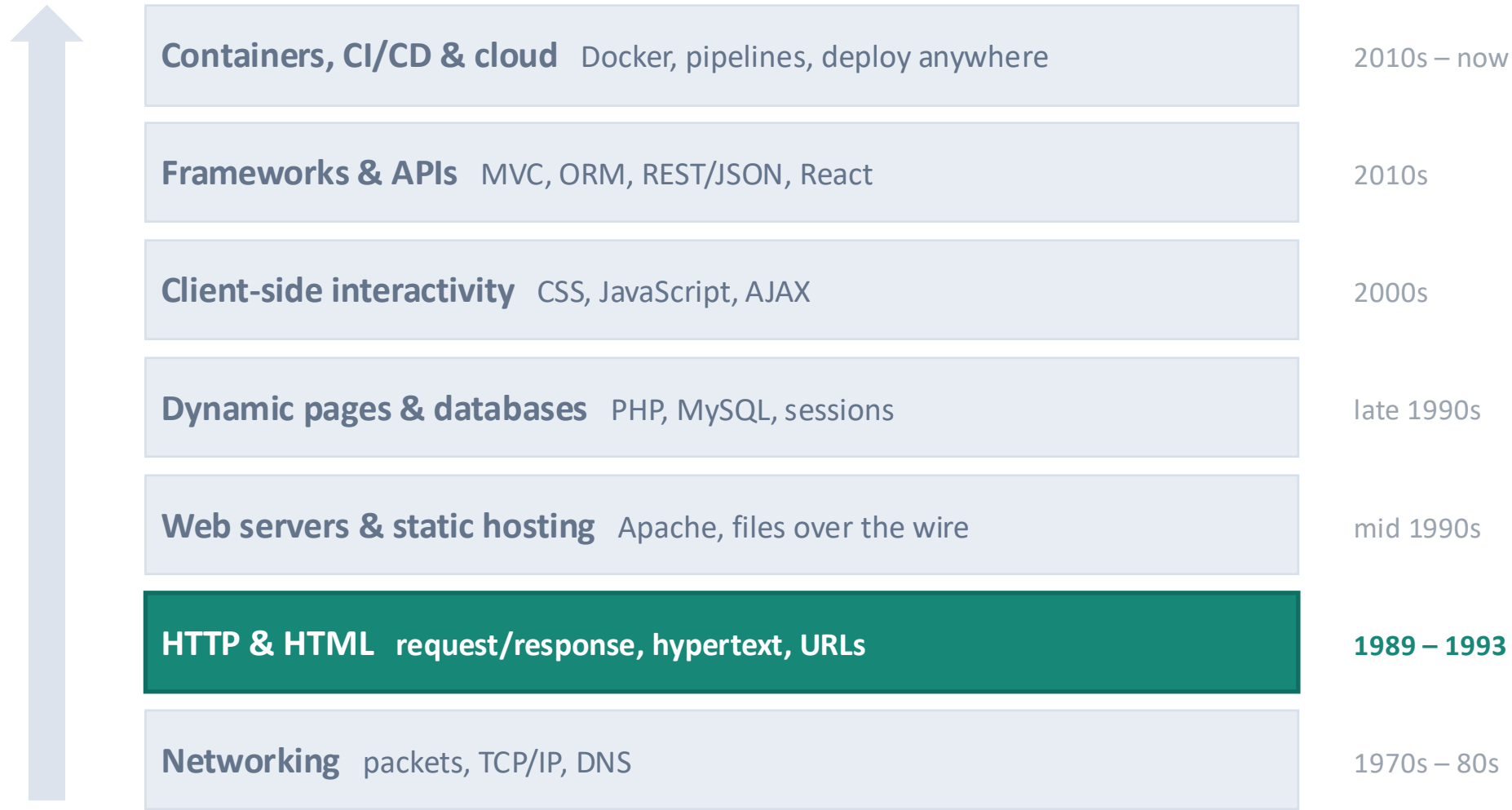
The bottom layer of the stack has been rebuilt four times — each time under a different pressure

1960s · ORIGINS	1970s – 80s · ONE INTERNET	1990s · OPENING UP	2000s – TODAY · SCALE
1961 – 64 Packet switching Baran and Davies split messages into independently routed packets. 1969 ARPANET Four nodes — UCLA, SRI, UCSB, Utah — on 50 kbit/s lines. 1970 NCP One host-to-host protocol for a single, centrally run network. The constraint Links were scarce and expensive; sharing them statistically was the point.	1974 TCP/IP Cerf and Kahn join dissimilar networks: dumb core, smart edges. 1983 Flag day ARPANET drops NCP for TCP/IP. Splitting TCP from IP also makes UDP possible. 1983 – 84 DNS A hand-edited HOSTS.TXT file gives way to delegated, hierarchical naming. 1980s Ethernet & LANs Cheap local networks multiply hosts and push addressing pressure outward.	1989 – 91 BGP and the Web Policy routing between networks; HTTP becomes the network's killer app. 1993 – 95 Going public NSFNET retires; ISPs and exchange points carry the traffic. 1993 – 94 CIDR and NAT Classful addressing abandoned, networks hidden behind one address. Driver: IPv4 scarcity. 1994 – 99 SSL / TLS Security arrives as a layer bolted on top, not as a redesign of IP.	1998 IPv6 128-bit addressing (RFC 2460; RFC 8200 in 2017) lifts the 32-bit ceiling. 2011 IPv4 exhaustion IANA's free pool empties. Address transfer markets and carrier-grade NAT follow. 2026 IPv6 majority March 2026: 50.1% of Google's users reached it over IPv6. Now Latency rules CDNs, HTTP/2 (2015) and HTTP/3 over QUIC (2022) cut round trips.

KEY IDEA · Every change came from a pressure, not a plan: survivability, then incompatibility, then address scarcity, now latency. IPv6 is the slowest transition because NAT made the old ceiling tolerable.

The web systems stack, built layer by layer

Each layer assumes the one below it — and each was invented to solve a problem the layer below created





Sir Tim Berners-Lee invented the World Wide Web in 1989.

Image: © CERN

nature

[Explore content](#) ▾ [About the journal](#) ▾ [Publish with us](#) ▾

[nature](#) > [commentaries](#) > [article](#)

Commentary | Published: 26 April 2001

Publishing on the semantic web

[Tim Berners-Lee](#) & [James Hendler](#)

[Nature](#) **410**, 1023–1024 (2001) | [Cite this article](#)

[Save article](#)

3902 Accesses | **229** Citations | **5** Altmetric | [Metrics](#)



COMPUTER SCIENCE

Acting Department Head of Cognitive Science, Tetherless World Senior Constellation
Professor of Computer, Web and Cognitive Science and Special Advisor to the Provost

✉ hendler@cs.rpi.edu

☎ [518-276-4401](tel:518-276-4401)

🌐 <https://www.cs.rpi.edu/~hendler/>

What Tim Berners-Lee actually invented

He did not invent the Internet; he designed the World Wide Web as a working system on top of it.

THE CORE WEB SYSTEM — CERN, 1989–1990

A set of standards plus working software

URI / URL
A universal address for each Web resource

HTTP
Rules for browser–server requests and responses

HTML
Linked documents structured with markup

WorldWideWeb
The first browser-editor; it could read and author pages

CERN httpd
The first Web server software, running on a NeXT computer

Together, these made the idea testable—not just a proposal.

DEPENDENCIES & LATER ADDITIONS

Already existed below the Web

- Internet and packet networks
- TCP/IP transport
- DNS host naming
- SGML and earlier hypertext ideas

Added by others afterward

CSS — proposed by Håkon Wium Lie in 1994; developed with Bert Bos
JavaScript — Brendan Eich, 1995
Web databases, frameworks and cloud — later ecosystem

Class takeaway

The invention was the architecture: address + protocol + document format + browser + server.

A short history of the web server

Serve a file → run a program → be the program: how the thing answering the request changed shape

1990 – 93 CERN httpd

- Tim Berners-Lee writes the first server on a NeXT cube
- Static files over HTTP; no programs, no state
- Released to the public domain in 1993

1993 NCSA httpd

- Rob McCool's server at NCSA, alongside Mosaic
- CGI lets a request run a program — the dynamic web
- Development stalls when he leaves; forks multiply

1995 Apache

- Eight developers merge their NCSA patches
- Modules (mod_php, mod_ssl) make it extensible
- The A in LAMP; the top server for two decades

1995 – 2004 IIS and nginx

- IIS ships with Windows Server; ASP follows
- Igor Sysoev writes nginx for the C10K problem
- Event loop replaces one process per connection

2009 Node.js

- Ryan Dahl puts V8 and a non-blocking loop on the server
- The application is the server — no Apache in front
- One language end to end; npm becomes the ecosystem

Today Proxies and platforms

- nginx or Cloudflare in front, app processes behind
- Containers and serverless functions carry the code
- nginx ~32%, Cloudflare ~29%, Apache ~23% (W3Techs)

KEY IDEA Each generation moved logic closer to the request — file, then program, then the program itself. What decided the winners was the concurrency model, not the feature list.

PHP: how it emerged, and why it is still everywhere

A weekend tool for tracking résumé visits became the language that answers most of the web's requests

HOW IT EMERGED

- **1994**
Rasmus Lerdorf releases “Personal Home Page Tools” — small C programs to log visits to his online résumé.
- **1995 – 1998**
PHP/FI adds forms and database access; Zeev Suraski and Andi Gutmans rewrite the parser, producing PHP 3.
- **2000**
PHP 4 ships on the Zend Engine. Cheap shared hosting makes LAMP the default way to put a site online.
- **2004**
PHP 5 brings real object orientation, exceptions and PDO — large applications become practical.
- **2015 – 2020**
PHP 7 roughly doubles performance; PHP 8 adds a JIT, union types, attributes and named arguments.

WHY IT IS STILL RELEVANT

~71%

server-side language
share

43%

of all sites run
WordPress

62%

of PHP sites run
PHP 8.x

- **Nothing else is close.** Ruby 6.7%, JavaScript 5.9%, Java 5.4%, ASP.NET 4.4% of sites with a known server-side language.
- **The installed base keeps growing.** Every WordPress, Drupal and Joomla site is a PHP site, carrying decades of content and plugins.
- **Modern PHP is a different language.** Typed properties, enums, attributes and a JIT — the 2005 criticisms mostly no longer apply.
- **Frameworks closed the gap.** Laravel and Symfony bring routing, ORM, testing and queues — the layer the 2000s version lacked.

KEY IDEA PHP survived by being the easiest thing to deploy — one file per URL, nothing to keep running between requests — and then grew up: types, a JIT, and frameworks that hold large codebases together.

Databases and ORM: tables and documents serve different needs

Relational databases enforce structure and joins; document databases keep related data together; ORM/ODM bridges both to application code

HOW THE DATA LAYER EVOLVED

- **1970**
E. F. Codd defines the relational model: data organized into tables, connected by keys and queried declaratively.
- **1986**
SQL becomes an ANSI standard, giving relational systems a common language for defining and querying data.
- **1995 – 1996**
MySQL and PostgreSQL make robust relational databases accessible to the rapidly growing web.
- **2009**
MongoDB launches and helps popularize JSON-like documents, flexible schemas and horizontal scaling.
- **2010s – now**
ORMs map tables and relationships; ODMs map document collections into application objects.

CHOOSING THE MODEL & MAPPER

RELATIONAL

tables, joins,
transactions

DOCUMENT

JSON-like records,
flexible schema

ORM / ODM

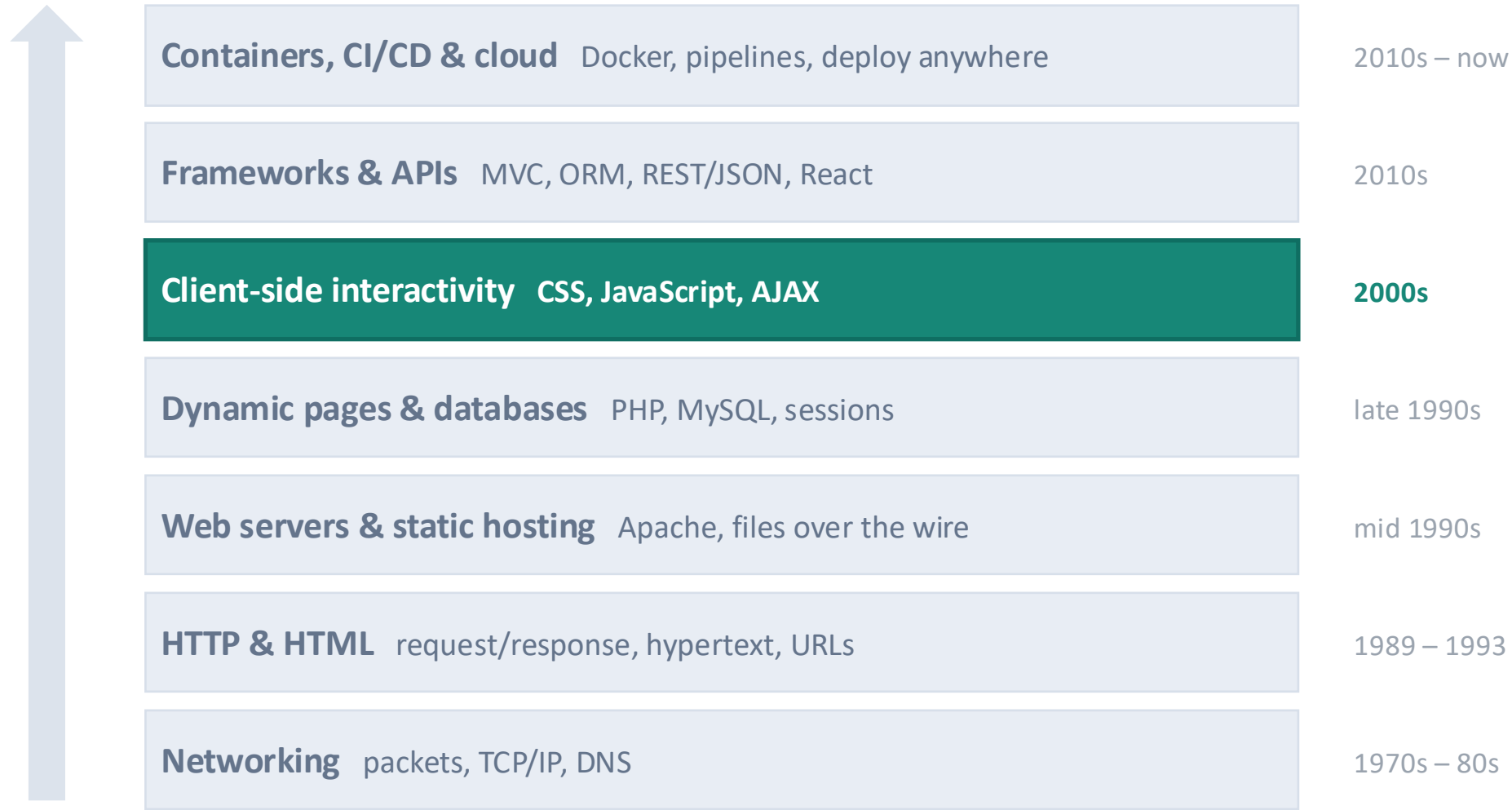
maps storage to objects

- **Choose relational for integrity.** Strong constraints, joins and transactions fit connected, structured data.
- **Choose documents for aggregates.** Related fields can travel together and evolve without many cross-table joins.
- **Match the mapper.** ORM targets rows and relations; ODM targets documents, collections and embedded data.
- **Abstraction has limits.** Indexes, query plans, consistency and data modeling still determine performance.

KEY IDEA Relational and document databases optimize different data shapes. Use ORM or ODM to reduce application plumbing — but design the underlying model and understand the generated queries.

The web systems stack, built layer by layer

Each layer assumes the one below it — and each was invented to solve a problem the layer below created



The story of CSS: separating content from presentation

“CSS ... took into account that on the Web, the style of a document couldn't be designed by either the author or the reader on their own, but that their wishes had to be combined, or cascaded, in some way.” — Bert Bos, W3C

WHY THE WEB NEEDED CSS

Early HTML mixed meaning and appearance

Authors used presentational tags, browser-specific extensions, and repeated formatting on every page.

HTML

headings
paragraphs
links
images

+

CSS

color
type
spacing
layout

One HTML document could now support different visual presentations—including user preferences and different media.

FROM PROPOSAL TO PLATFORM



1994

The proposal

Håkon Wium Lie proposes Cascading HTML Style Sheets at CERN.



1995

Ideas converge

Lie and Bert Bos combine their proposals and develop CSS through the W3C.



1996

CSS Level 1

The first W3C Recommendation standardizes fonts, colors, spacing and basic layout.



1998+

CSS2 → modules

Positioning and media support expand; later work evolves as independent modules—not one monolithic “CSS3.”

THE KEY INVENTION: THE CASCADE

Browser defaults



User preferences



Author styles

+ specificity + source order



Computed style

Quote: Bert Bos, “A brief history of CSS until 2016,” W3C — [w3.org/Style/CSS20/history.html](https://www.w3.org/Style/CSS20/history.html)

How CSS changed: from document styling to a complete UI system

By 2026, CSS controls responsive layout, reusable design systems, component context, animation, color, and increasingly application-like experiences.

FIVE MAJOR ERAS OF CHANGE

The Web platform accumulated capabilities; older CSS still works underneath.

1996–1998	2000s	2010–2018	2016–2022	2022–2026
FOUNDATION	STANDARDIZATION	LAYOUT & MOTION	DESIGN SYSTEMS	CONTEXT-AWARE CSS
• Fonts, colors and backgrounds • Box model and spacing • Positioning and media types • CSS1, then CSS2	• CSS2.1 clarified interoperable behavior • Better selectors and generated content • Standards-mode browsers reduced vendor differences	• Media queries enabled responsive design • Transforms, transitions and animation • Web fonts • Flexbox and Grid replaced layout hacks	• Custom properties (variables) • Logical properties for writing modes • Subgrid • Cascade layers, :is(), :where() and :has()	• Container and style queries • Native nesting • Modern color and math functions • View transitions, scoping and richer component styling

THE BIG SHIFTS

1

Separation

HTML describes meaning; CSS controls presentation.

2

Layout

From floats and tables to Flexbox and Grid.

3

Context

Styles now respond to viewport, container, state and user settings.

Important: “CSS3” was never one finished release. CSS evolved into independently versioned modules, documented together in periodic CSS Snapshots.

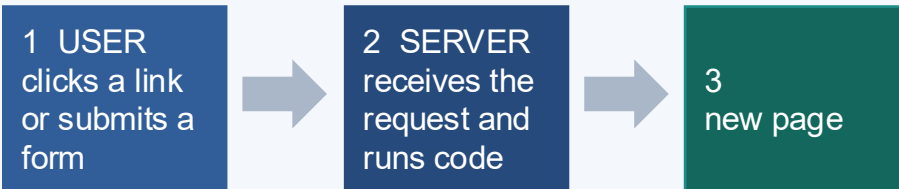
Sources: W3C — CSS1 and CSS2 Recommendations; CSS Snapshot 2026: CSS Cascading and Inheritance, Grid, Flexbox, Conditional Rules, Nesting and View Transitions specifications.

Why JavaScript emerged: the Web needed behavior, not just documents

HTML could describe a page and CSS could style it—but the browser still could not react intelligently after the page loaded.

THE PROBLEM IN THE EARLY WEB

Every interaction depended on the server



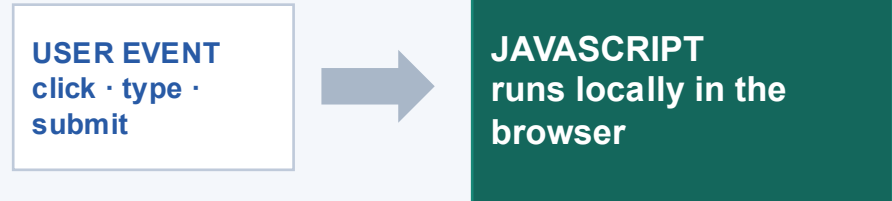
The browser had to wait for a round trip and usually replace the whole document.

What was difficult

- Immediate feedback
- Validate forms before submission
- Change page content after loading
- Respond to user and browser events

THE MISSING CLIENT-SIDE LAYER

JavaScript put small programs inside the browser



The script could inspect the page, calculate, validate input and change what the user saw—without requesting a new document for every action.

Design goal

A lightweight, approachable scripting language that complemented Java and was embedded directly in Netscape Navigator.

1995: THE FIRST USE CASES

1

Validate

Catch missing or invalid form fields before sending data.

2

Respond

Run code when the user clicks, types, loads or navigates.

3

Change

Update page content, browser status and interface behavior.

Key idea: JavaScript filled the gap between static documents and fully interactive software in the browser.

Sources: Ecma International — “A brief history of ECMAScript”; MDN Web Docs — JavaScript overview and client-side scripting history. Accessed August 26, 2026.

The key history of JavaScript

From a 10-day prototype to a yearly standard that runs on every device

1995 – 1999 BORN IN A HURRY

1995

Brendan Eich writes the language in about ten days at Netscape; it ships in Navigator 2.0 as JavaScript.

1996

Microsoft ships JScript in Internet Explorer 3 — two dialects, one web, endless browser checks.

1997

Handed to ECMA for standardization: ECMAScript 1 (ECMA-262) becomes the official spec.

1999

ES3 adds regular expressions, try/catch and better string handling — the baseline for a decade.

2000 – 2008 THE AWKWARD YEARS

2005

AJAX: XMLHttpRequest lets pages update without reloading. Gmail and Maps show what is possible.

2006

jQuery papers over browser differences; DOM work becomes tolerable for ordinary developers.

2008

Google's V8 brings just-in-time compilation — JavaScript stops being slow.

ES4, abandoned

The ambitious rewrite collapses without consensus; the committee restarts with smaller steps.

2009 – TODAY STANDARD EVERYWHERE

2009

ES5 lands strict mode, JSON and property descriptors. Node.js takes JavaScript to the server.

2015

ES2015 (ES6) is the big one: let and const, classes, modules, promises, arrow functions.

2016 onward

TC39 ships yearly: async/await in 2017, optional chaining in 2020, steady additions since.

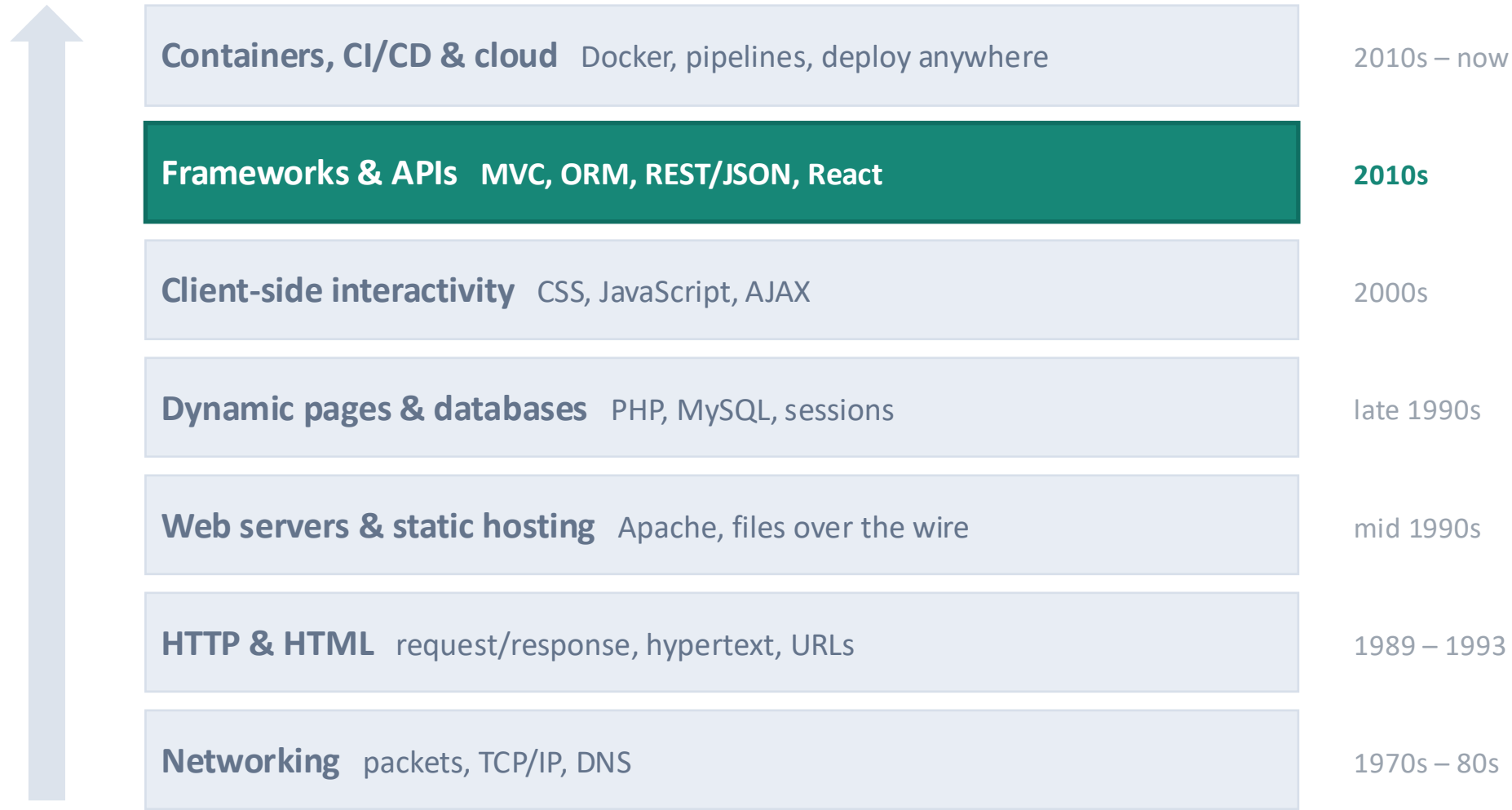
Today

One language across browser, server, edge and mobile — plus TypeScript and a deep tooling layer.

KEY IDEA JavaScript went from one vendor's scripting hack to a standard with a yearly release train — which is why the language you learn today keeps growing without breaking the web.

The web systems stack, built layer by layer

Each layer assumes the one below it — and each was invented to solve a problem the layer below created



What is a framework, and what problems does it solve?

Every web app repeats the same plumbing — a framework is that plumbing, written once, with opinions

THE PROBLEMS EVERY APP REPEATS

- Map a URL to the right code (routing)
- Talk to the database without writing raw SQL everywhere
- Render HTML without string-gluings (templates, escaping)
- Validate input; manage sessions and logins
- Stop XSS, CSRF and injection — every single time
- Decide where files go, so any teammate can navigate

WHAT A FRAMEWORK GIVES YOU

- Conventions — decisions you no longer have to make
- Batteries: router, ORM, templates, auth, test harness
- Security defaults written by people who got burned
- An upgrade path and a package ecosystem
- A shared vocabulary — new hires already know the layout
- Testability: seams for mocking and fixtures built in

Tailwind kills bespoke CSS architecture — utilities instead of naming systems

React kills manual DOM bookkeeping — describe the UI, it updates it

Laravel kills routing, ORM and auth boilerplate — one convention for all of it

KEY IDEA A framework is crystallized experience — thousands of teams' answers to the same six problems, packaged with opinions. The trade: you accept its conventions and learning curve, and in exchange you don't rebuild (or mis-build) the plumbing.

Frameworks arrive: CSS, JavaScript, PHP

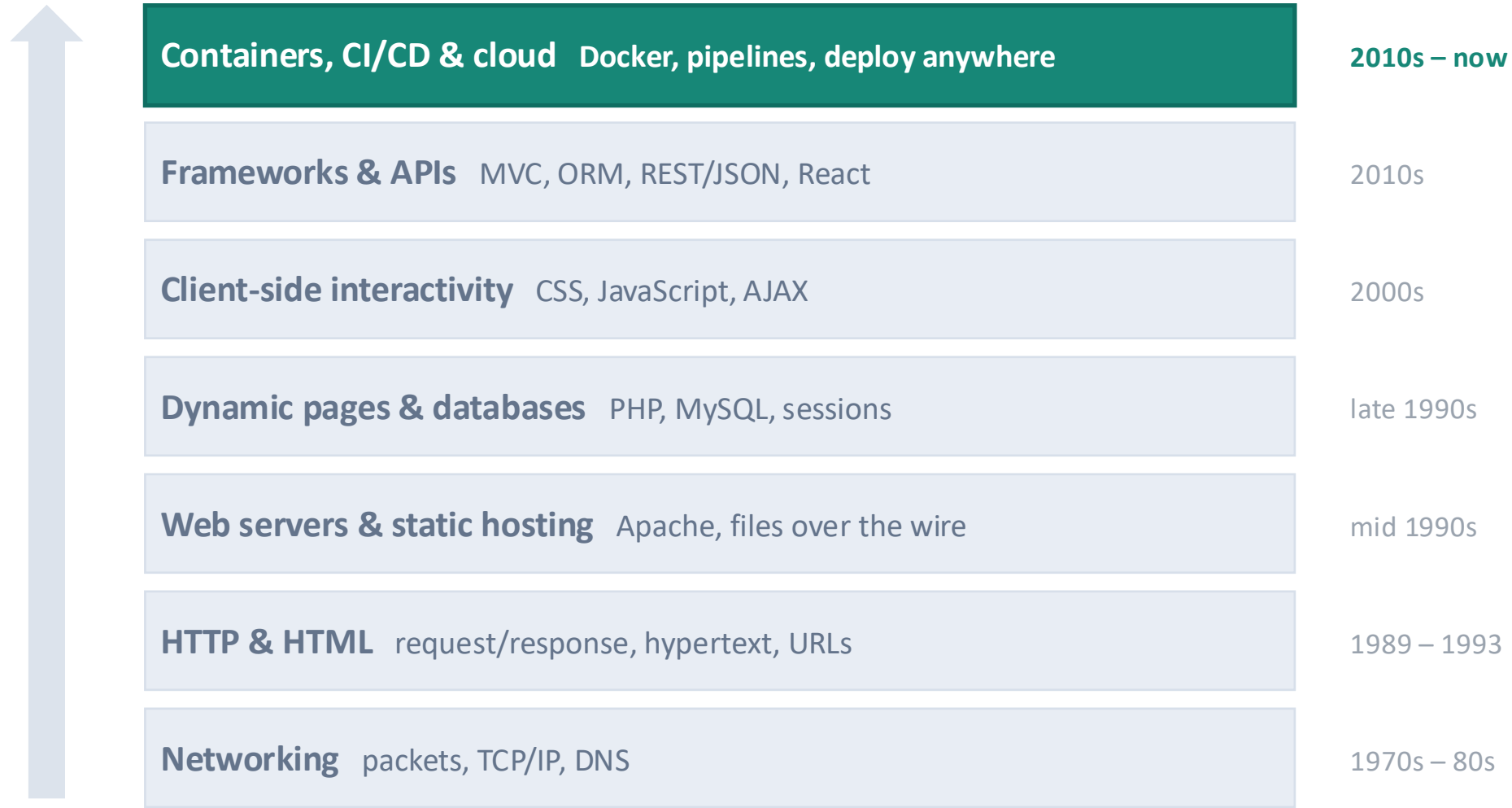
Every layer grew frameworks once its pain became universal — and they all arrived in the same dozen years



KEY IDEA Each lane frameworkized the same way: preprocessors and grids for CSS, DOM abstraction then components for JS, routing and ORM boilerplate for PHP. When you meet Tailwind, React and Laravel this semester, you are meeting the survivors of this timeline.

The web systems stack, built layer by layer

Each layer assumes the one below it — and each was invented to solve a problem the layer below created



What problem do containers, CI/CD and the cloud solve?

The top of the stack does to operations what frameworks did to code: it turns handcraft into files in your repo

THE PROBLEMS EVERY TEAM HITS

- “Works on my machine” — every laptop and server is subtly different
- Setup lives in a README, or in one person's head
- Environments drift: laptop $\neq$ staging $\neq$ production
- Deploys are manual, rare and scary — release day is an event
- A bad release is hard to undo
- Scaling means guessing demand and racking servers ahead of it

WHAT THE TOOLING GIVES YOU

- Docker: the environment is a versioned artifact — build once, run anywhere
- Infrastructure as code: servers described in files, reviewed like code
- CI: every push is built and tested automatically
- CD: deploys become boring — small, frequent, reversible
- Rollback = redeploy yesterday's image
- Cloud: capacity is an API call — pay for what you use, scale on demand

We will deal with Docker in Lab 1.
Testing is shortly after.

Every push CI runs your test suite before a human ever looks

The payoff the image you tested is the image that ships

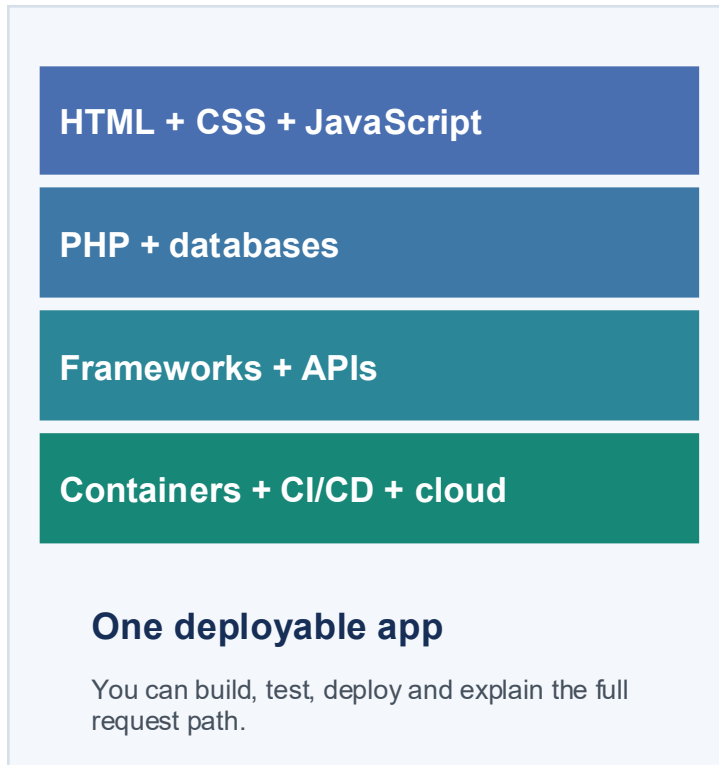
KEY IDEA The same move as frameworks, one level up: the environment, the release process and the server fleet stop being handcraft and become code — versioned, reviewed, testable and reproducible by anyone who clones the repo.

There are so many
more things to learn.

So Much More

The Web stack is the foundation. Modern systems add new ways to store data, coordinate work, divide software, place computation and delegate tasks to agents.

FALL Web Systems



WHAT
COMES
NEXT?

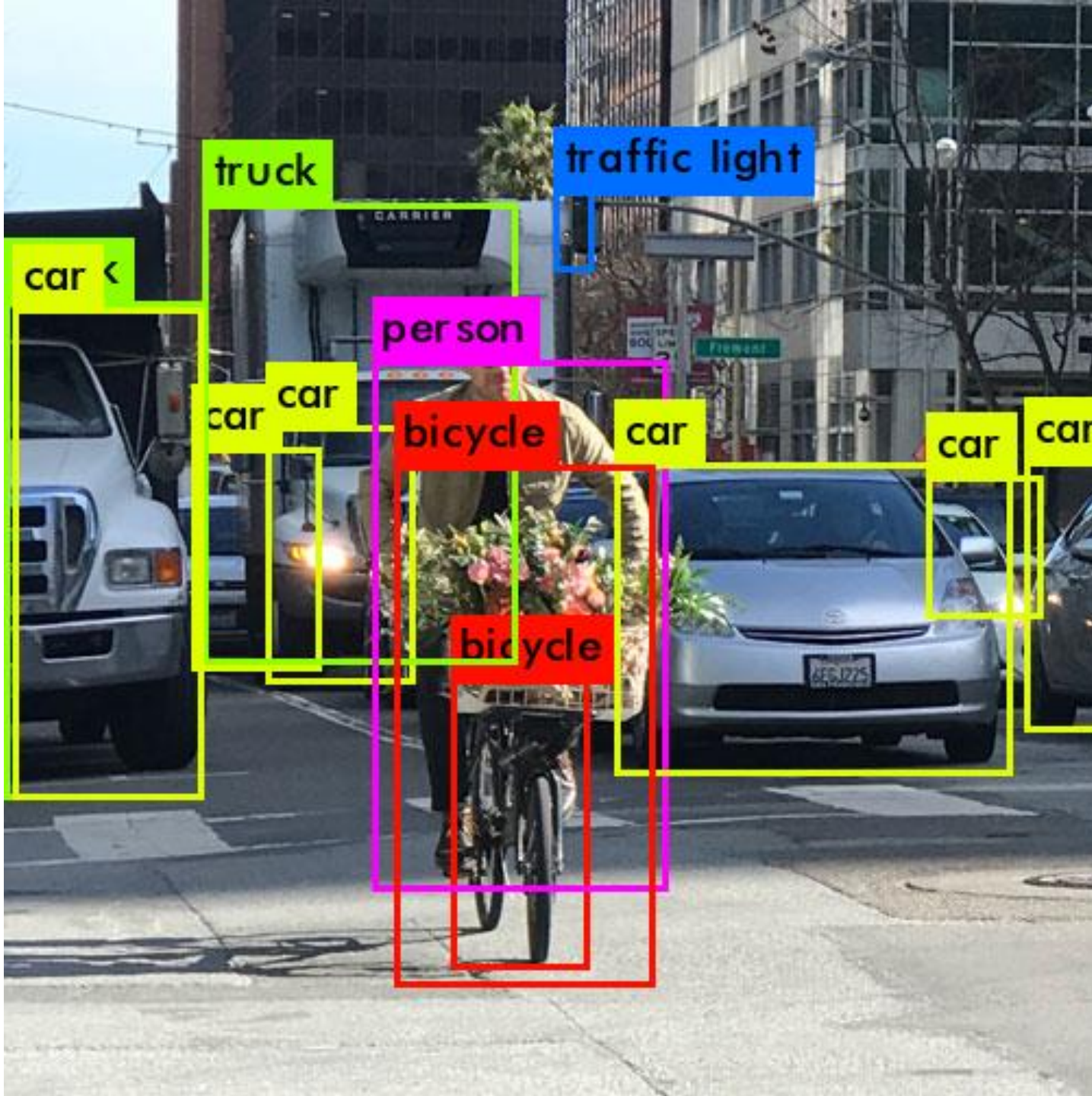
SPRING Advanced Web Systems

- **DATA MODEL** **Document databases**
How do we store nested, rapidly changing data when rigid tables are not the best fit?
- **COORDINATION** **Event-driven architectures**
How do parts of a system react asynchronously to messages, streams and real-world events?
- **BOUNDARIES** **Microservices**
When should one application become independently deployed services—and what complexity follows?
- **EXECUTION** **Cloud functions**
What if code runs on demand for a request, schedule or event instead of on a permanent server?
- **LOCATION** **Edge services**
How do we move computation, caching and decisions closer to users and connected devices?
- **AUTONOMY** **Agentic systems**
How do models use tools, memory and feedback loops to pursue goals while remaining observable and controlled?

THE NEXT MENTAL MODEL

From building one application → designing a distributed system that expects latency, partial failure and independent change.

We have reached a
new stage in the
evolution of our
ability to harness AI
as part of systems.

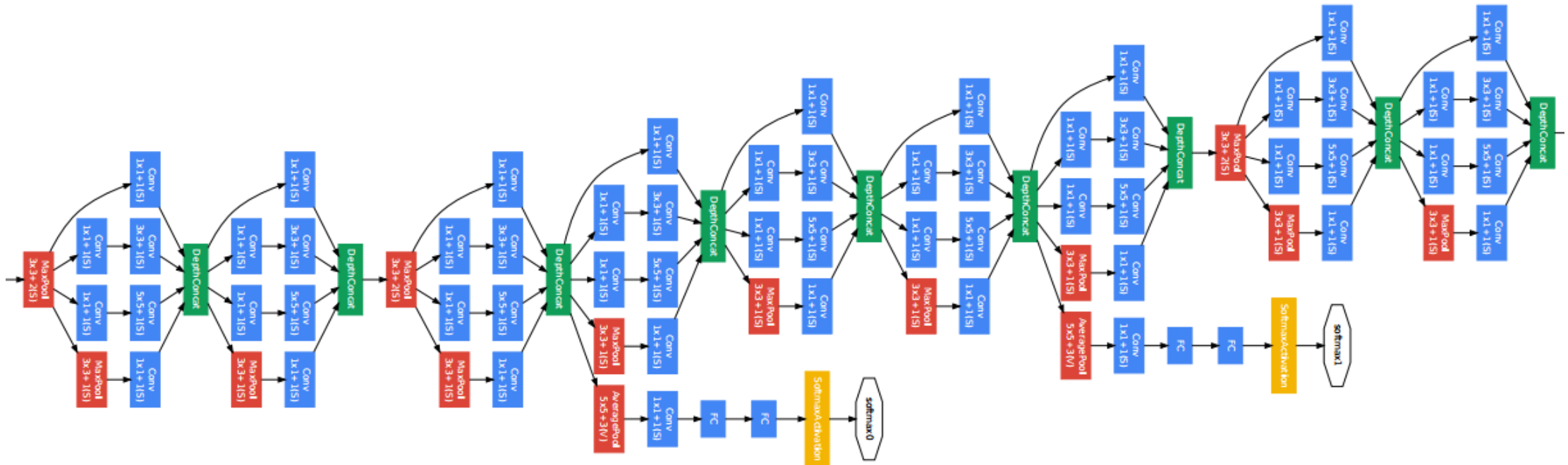


By learning
to harness
the
functions
created by
others, you
to can do
amazing
things

Neural Networks

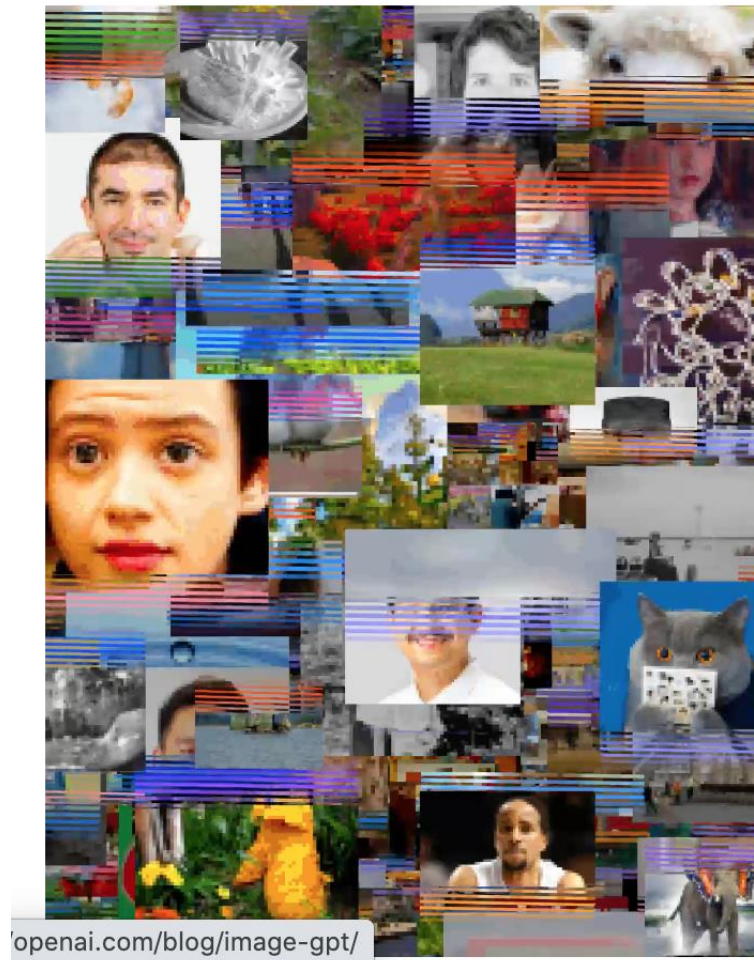
$$f(g(h(x)))$$

Modern neural networks are complex structures,
functions of functions of functions



General Algorithm (Transformer) Excels in Both Pixels and Text

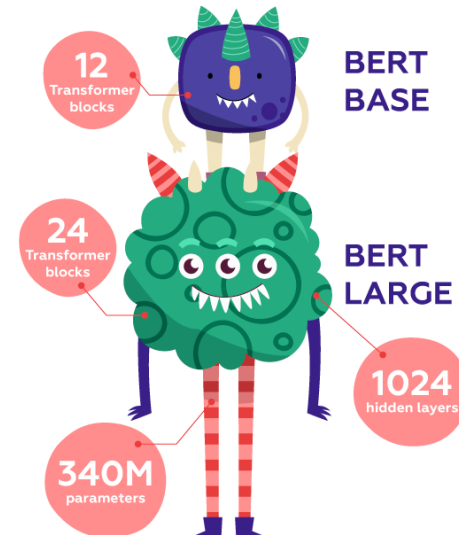
Image GPT



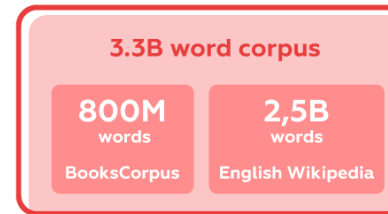
sciforce

BERT model at a glance

BERT comes in two sizes: BERT BASE, comparable to the OpenAI Transformer and BERT LARGE – the model which is responsible for all the striking results.



BERT is pre-trained on 40 epochs over:

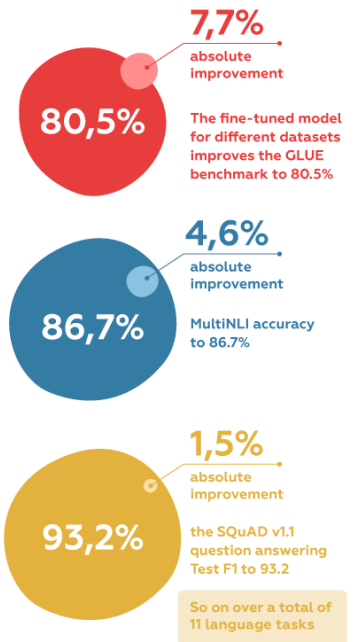


INPUT

BERT takes a sequence of words which keep flowing up the stack. Each layer applies self-attention, and passes its results through a feed-forward network, and then hands it off to the next encoder.

OUTPUT

The output of each position is a vector of size called `hidden_size` (768 in BERT Base). This vector can be used as the input for a classifier you choose.



https://miro.medium.com/max/2966/0*-EkGos68wndYtgnH

Neural Networks – Functions all the way down.

$f(g(h(x)))$

Modern neural networks are complex structures, functions of functions of functions.



https://en.wikipedia.org/wiki/Turtles_all_the_way_down

"AlphaGo is almost like the God of Go"

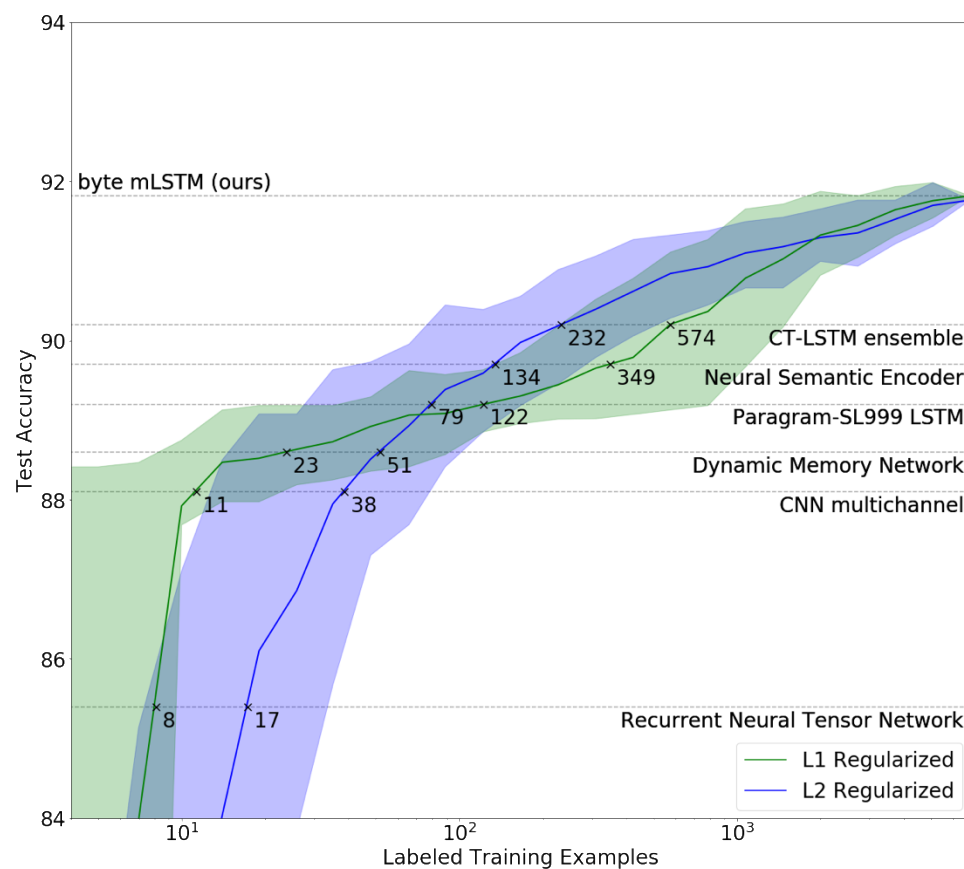
AlphaGo's move
37 described as
"So beautiful."





<https://openai.com/blog/solving-rubiks-cube/>

Unsupervised Sentiment Neuron Beat Supervised Sentiment Analysis



OpenAI developed an unsupervised system which learns an excellent representation of sentiment, despite being trained only to predict the next character in the text of Amazon reviews.

<https://blog.openai.com/unsupervised-sentiment-neuron/>

What does it mean to
be a web developer
today in an age of AI?

How AI Is Impacting Web Development

- **Role Change From Coding To Oversight:** AI generates much of the boilerplate and routine code, so developers increasingly "supervise, double check and verify AI" rather than write every line themselves.
- **Productivity Gains, Not Full Replacement:** AI delivers large productivity improvements for many tasks, especially repeatable UI/code generation, but it doesn't remove the need for human judgment and ownership.
- **Bottleneck Shift To Architecture & Ownership:** The main remaining human problems are architecture, system design, integrations, security, and long-term ownership — AI shifts the bottleneck from writing code to ensuring maintainability.

Skills To Prioritize

- **System Design & Architecture:** Senior judgment about boundaries, decoupling, and long-term structure becomes more valuable than typing out components.
- **Code Review & Correctness:** Reading and correcting AI output is a core skill — catching subtle coupling, unnecessary complexity, or security holes.
- **Security, Testing & Compliance:** AI can introduce vulnerabilities or non-compliant solutions; humans must own audits, testing, and regulatory requirements.
- **Platform, Infra & DevOps:** Ownership of infra, monitoring, CI/CD and deployments remains critical as AI accelerates shipping but not reliable operations.
- **Product & Communication Skills:** Understanding business requirements, proposing tradeoffs, and communicating scenarios are essential as AI lowers the entry barrier to code.

Summary: The Web is a layered system that keeps evolving

- **Internet foundation:** The Internet provides the network; the Web added URLs, HTTP, HTML, browsers and servers.
- **Layered experience:** CSS separated presentation from structure; JavaScript made pages interactive.
- **Dynamic applications:** PHP and databases moved the Web from static documents to data-driven systems.
- **Modern delivery:** Frameworks, APIs, containers, CI/CD and cloud improved integration, deployment and operations.
- **What comes next:** document databases, event-driven architectures, microservices, cloud functions, edge services and agentic systems.

It's an exciting time to build systems and be a web developer!

Before Next Class

- Fill out the class survey.
- Download Github Desktop.
- Clone Class Repo.
 - See Readings on Docker
- Install Docker
- An invitation to a private repo for labs/homeworks will be provided by Tuesday.

Reading

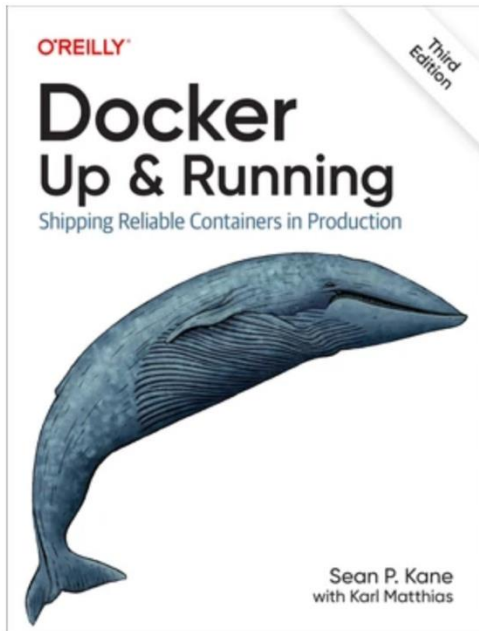


BOOK

Docker: Up & Running, 3rd Edition

★★★★★ [1 review](#)

By [Sean P. Kane](#), [Karl Matthias](#)



TIME TO COMPLETE:
11h 17m

LEVEL:
Intermediate to advanced

SKILLS:
[Docker](#)

PUBLISHED BY:
[O'Reilly Media, Inc.](#)

PUBLICATION DATE:
April 2023

PRINT LENGTH:
418 pages

[+ Add to playlist](#)

- Available [through the library](#)
- Chapters 1, 2, 4, 5, 6 (through getting inside a running container)

Start